

OSD-to-OpenSearch API Mapping

Generated: 2026-03-19

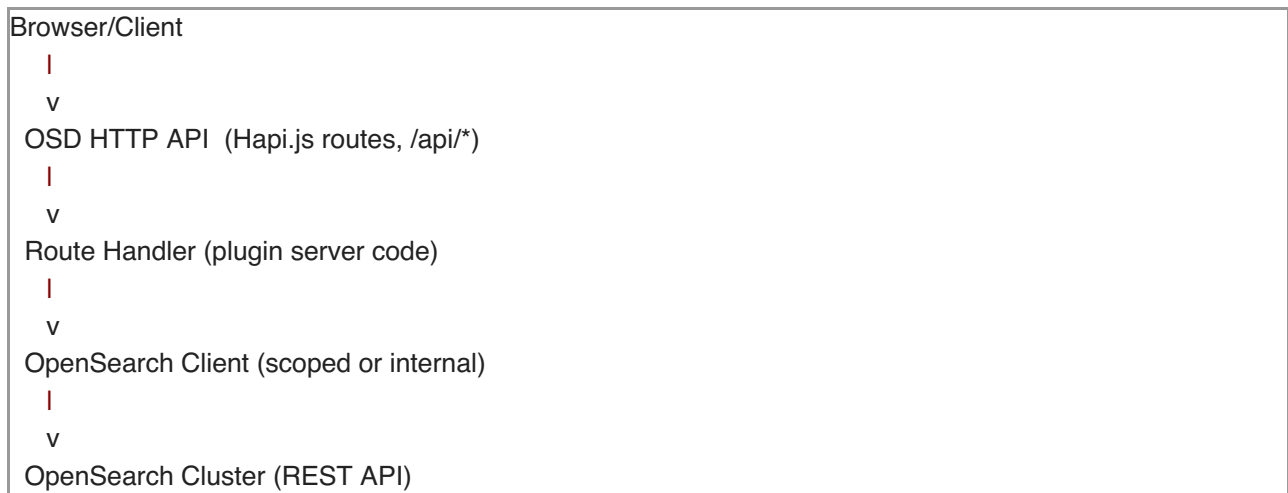
Sources: OPENSEARCH_CLIENT_USAGE.json, OSD_API_ROUTES.json, codebase analysis

This document maps OpenSearch Dashboards (OSD) HTTP API endpoints to the underlying OpenSearch APIs they invoke.

Table of Contents

1. [Architecture Overview](#)
 2. [Client Access Patterns](#)
 3. [Core: Saved Objects API](#)
 4. [Core: Status & Health](#)
 5. [Plugin: Data \(Search\)](#)
 6. [Plugin: Console \(Dev Tools Proxy\)](#)
 7. [Plugin: Data Importer](#)
 8. [Plugin: Application Config](#)
 9. [Plugin: Region Map](#)
 10. [Plugin: Index Pattern Management](#)
 11. [Plugin: Query Enhancements](#)
 12. [Plugin: Data Source Management](#)
 13. [Plugin: Chat \(ML/AI\)](#)
 14. [Plugin: Workspace](#)
 15. [Plugin: Home \(Sample Data\)](#)
 16. [Plugin: Telemetry](#)
 17. [Plugin: Vis Type Timeline](#)
 18. [Plugin: Vis Type Timeseries \(TSVB\)](#)
 19. [Plugin: Data Source](#)
 20. [Plugin: Saved Objects Management](#)
 21. [Plugin: Share \(Short URLs\)](#)
 22. [Plugin: Legacy Export](#)
 23. [Core: Dynamic Config Store \(Internal\)](#)
 24. [Core: Saved Objects Migrations \(Internal\)](#)
 25. [Cross-Reference: OpenSearch API to OSD Endpoints](#)
 26. [Summary Statistics](#)
-

Architecture Overview



OSD acts as a gateway: browser clients call OSD's HTTP API, and route handlers translate these into one or more OpenSearch REST API calls. The mapping is not always 1:1 -- a single OSD endpoint may call multiple OpenSearch APIs, and some OpenSearch APIs are called from internal background processes rather than HTTP routes.

Client Access Patterns

Pattern	Description	Used By
context.core.opensearch.client.asCurrentUser	Scoped client with user auth headers	Most route handlers
context.core.opensearch.client.asInternalUser	Internal client (no user auth)	Background tasks, config reads
opensearch.client.asInternalUser	Core-level internal client	Health checks, migrations
context.core.opensearch.legacy.client.callAsCurrentUser	Legacy string-based endpoint call	Older plugins (deprecated)
client.transport.request({ method, path })	Low-level HTTP for custom OpenSearch plugin APIs	Console, Chat, Query Assist
context.dataSource.opensearch	Multi-data-source scoped client	Data Source Management (MDS)

Core: Saved Objects API

The saved objects API is the **largest consumer** of OpenSearch APIs. It manages all persistent objects (dashboards, visualizations, index patterns, searches, etc.).

Source: src/core/server/saved_objects/routes/
OpenSearch Client: src/core/server/saved_objects/service/lib/repository.ts

Endpoint Mappings

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
			Query params -> OpenSearch query DSL

/api/saved_objects/_find	GET	search	-> search on .kibana index
/api/saved_objects/{type}/{id}	GET	get, mget	Fetches single saved object by type+id from .kibana index
/api/saved_objects/_bulk_get	POST	mget	Batch fetch of saved objects by type+id pairs
/api/saved_objects/{type}/{id?}	POST	index or create	Creates new saved object. Uses create (no overwrite) or index (with overwrite flag)
/api/saved_objects/{type}/{id}	PUT	update	Partial update of saved object document
/api/saved_objects/{type}/{id}	DELETE	mget + delete / update	Checks doc exists via mget, then deletes or removes namespace
/api/saved_objects/_bulk_create	POST	mget + bulk	Pre-checks existing docs via mget, then bulk indexes
/api/saved_objects/_bulk_update	PUT	mget + bulk	Pre-checks docs via mget, then bulk updates
/api/saved_objects/_import	POST	mget + bulk + create	Imports saved objects from NDJSON stream
/api/saved_objects/_export	POST	search (with scroll)	Exports saved objects as NDJSON using scroll API
/api/saved_objects/_resolve_import_errors	POST	mget + bulk	Resolves conflicts from import
/internal/saved_objects/_migrate	POST	(all migration APIs)	Triggers saved objects migrations

Request/Response Transformations

OSD Request: **GET** /api/saved_objects/_find?type=dashboard&search=test

|

v

Repository.find() builds OpenSearch query:

```
{
  index: ".kibana",
  body: {
    query: { bool: { filter: [...type filters, ...search query] } },
    sort: [...],
    size: perPage,
    from: (page - 1) * perPage
  }
}
```

|

v

OpenSearch: **POST** /.kibana/_search

|

v

OSD Response: { saved_objects: [...], total, per_page, page }

Core: Status & Health

Source: src/core/server/opensearch/version_check/ensure_opensearch_version.ts,
src/core/server/status/routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/status	GET	(reads cached status)	Serves pre-computed status from background polling
(internal background)	-	nodes.info	Checks OpenSearch node versions for compatibility
(internal background)	-	cluster.state	Gets cluster node IDs for optimized health check
(internal background)	-	cat.plugins	Cross-compatibility service checks installed OpenSearch plugins
(internal background)	-	info()	Core usage data service gets cluster info

Plugin: Data (Search)

Source: src/plugins/data/server/search/, src/plugins/data/server/autocomplete/,
src/plugins/data/server/index_patterns/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/internal/search/{strategy}/{id?}	POST	search	User search queries -> OpenSearch search API with abort signal support
/internal/search/{strategy}/{id}	DELETE	(cancel search)	Cancel in-flight search
/internal/_msearch	POST	msearch	Multi-search for batch query execution (deprecated)
/api/opensearch-dashboards/suggestions/values/{index}	POST	search (legacy)	Autocomplete value suggestions
/api/index_patterns/_fields_for_wildcard	GET	Legacy client field caps	Get fields for wildcard index pattern
/api/index_patterns/_fields_for_time_pattern	GET	Legacy client field caps	Get fields for time-based pattern
/api/opensearch-dashboards/scripts/languages	GET	(none)	Returns supported script languages

Request/Response Transformations (Search)

```

OSD Request: POST /internal/search/opensearch
  body: { params: { index: "my-index-*", body: { query: {...}, aggs: {...} } } }
  |
  v
opensearch_search_strategy.ts:
  const params = shimHitsTotal(request.params);
  shimAbortSignal(client.search(params), options?.abortSignal)
  |
  v
OpenSearch: POST /my-index-*/_search
  body: { query: {...}, aggs: {...} }
  |
  v
OSD Response: { rawResponse: { hits: {...}, aggregations: {...} } }

```

Plugin: Console (Dev Tools Proxy)

Source: `src/plugins/console/server/routes/api/console/proxy/`

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
<code>/api/console/proxy</code>	POST	<code>transport.request</code> (ANY)	Proxies arbitrary user requests directly to OpenSearch
<code>/api/console/opensearch_config</code>	GET	(none)	Returns OpenSearch config for console UI
<code>/api/console/api_server</code>	GET/POST	(none)	API spec definitions for autocomplete

Request/Response Transformations

```

OSD Request: POST /api/console/proxy?path=/_cat/indices&method=GET
  |
  v
create_handler.ts:
  client.transport.request({
    method: req.query.method,
    path: req.query.path,
    body: req.payload,
    querystring: req.query.query_string
  })
  |
  v
OpenSearch: GET /_cat/indices (or ANY user-specified endpoint)
  |
  v
OSD Response: Raw OpenSearch response (passthrough)

```

Note: The Console proxy is a passthrough -- it can call ANY OpenSearch API. This is the most powerful and potentially dangerous endpoint.

Plugin: Data Importer

Source: src/plugins/data_importer/server/routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/data_importer/_import_file	POST	indices.exists + indices.create + index + indices.updateAliases	Imports CSV/JSON/NDJSON files into new index
/api/data_importer/_import_text	POST	indices.exists + indices.create + index + indices.updateAliases	Imports text content into new index
/api/data_importer/_preview	POST	indices.exists + indices.getMapping	Previews import with schema detection
/api/data_importer/_cat_indices	GET	cat.indices	Lists available indices

Request/Response Transformations (Import)

```
OSD Request: POST /api/data_importer/_import_file
  body: { file: <multipart>, index: "my-data", dataSourceId: "..." }
  |
  v
import_file.ts:
  1. client.indices.exists({ index }) // Check if index exists
  2. client.indices.create({ index, body: { mappings, settings } }) // Create if needed
  3. For each row/record:
    processor.index({ index, body: record }) // Index documents
  4. client.indices.updateAliases(...) // Optional alias creation
  |
  v
OpenSearch: PUT /my-data (create index)
            PUT /my-data/_doc/{id} (index each document)
            POST /_aliases (alias management)
  |
  v
OSD Response: { success: true, count: N, errors: [...] }
```

Plugin: Application Config

Source: src/plugins/application_config/server/routes/,
src/plugins/application_config/server/opensearch_config_client.ts

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/appconfig/{entity}	GET	get	Fetches config entity by ID (uses asInternalUser)
/api/appconfig/{entity}	POST	index	Creates/updates config value (uses asCurrentUser)
/api/appconfig/{entity}	DELETE	delete	Deletes config entity (uses asCurrentUser)
/api/appconfig	GET	search	Lists all config entities (uses asInternalUser)

Plugin: Region Map

Source: src/plugins/region_map/server/routes/opensearch.ts

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/geospatial/_indices	GET	cat.indices	Lists indices for geo data selection
/api/geospatial/_search	POST	search	Searches for geo data in specified index
/api/geospatial/_mappings	GET	indices.getMappings	Gets index mappings to find geo fields

Plugin: Index Pattern Management

Source: src/plugins/index_pattern_management/server/routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/internal/index-pattern-management/preview_scripted_field	POST	search	Previews scripted field by running search with script_fields
/internal/index-pattern-management/resolve_index/{query}	GET	indices.resolveIndex	Resolves index pattern to concrete indices

Request/Response Transformations

OSD Request: **POST** /internal/index-pattern-management/preview_scripted_field

body: { indexPatternTitle, script, lang, additionalFields }

|

v

preview_scripted_field.ts:

```
client.search({
  index: indexPatternTitle,
  body: {
    query: { match_all: {} },
    size: 10,
    _source: additionalFields,
    script_fields: { preview: { script: { lang, source: script } } }
  }
})
```

|

v

OpenSearch: **POST** /{indexPatternTitle}/_search

|

v

OSD Response: { hits: [...with script field values...] }

Plugin: Query Enhancements

Source: src/plugins/query_enhancements/server/routes/

OSD Endpoint	Method	OpenSearch APIs Called
/api/enhancements/assist/generate	POST	transport.request -> /_plugins/_ml/agents/{id}/_execute

/api/enhancements/assist/languages	GET	(via agent)
/api/enhancements/datasource/connections/{id?}	GET	transport.request -> /_plugins/_query/_datasources
/api/enhancements/datasource/async_jobs	GET/POST/DELETE	transport.request -> direct que
/api/enhancements/datasource/remote_clusters/list	GET	transport.request
/api/enhancements/datasource/remote_clusters/indexes	GET	transport.request
PPL/SQL search strategies	POST	Legacy callAsCurrentUser('enhancem

Request/Response Transformations (Query Assist)

```

OSD Request: POST /api/enhancements/assist/generate
  body: { question: "show me sales by region", index: "sales-*", language: "PPL" }
  |
  v
agents.ts:
  client.transport.request({
    method: 'POST',
    path: `/_plugins/_ml/agents/${agentId}/_execute`,
    body: { parameters: { question, index } }
  })
  |
  v
OpenSearch: POST /_plugins/_ml/agents/{agentId}/_execute
  body: { parameters: { question: "...", index: "sales-*" } }
  |
  v
OSD Response: { query: "source=sales-* | stats count() by region" }

```

Plugin: Data Source Management

Source: src/plugins/data_source_management/server/routes/

DSL Routes

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/dsl/_search	POST	Legacy callAsCurrentUser('search')	Execute DSL search query

/api/dsl/_cat	GET	cat.indices	Get catalog of indices
/api/dsl/_mapping	GET	indices.getMapping	Get index mappings
/api/dsl/_settings	GET	indices.getSettings	Get index settings
/api/dsl/_cat/dataSourceMDSId={id}	GET	cat.indices (via MDS client)	Get catalog for remote data source
/api/dsl/_mapping/dataSourceMDSId={id}	GET	indices.getMapping (via MDS client)	Get mappings for remote data source
/api/dsl/_settings/dataSourceMDSId={id}	GET	indices.getSettings (via MDS client)	Get settings for remote data source

PPL Routes

OSD Endpoint Method OpenSearch APIs Called Data Flow

/api/ppl/_search POST Legacy facet ppl.pplQuery Execute PPL query

Data Connection Routes

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/ppl/dataconnections/{name}	GET	Legacy ppl.getDataConnectionByld	Get data connection by name
/api/ppl/dataconnections/{name}	DELETE	Legacy ppl.deleteDataConnection	Delete data connection
/api/ppl/dataconnections/{name}	PUT	Legacy ppl.modifyDataConnection	Update data connection
/api/ppl/dataconnections	GET	Legacy ppl.getDataConnections	List all data connections

Async Query Jobs Routes

OSD Endpoint	Method	OpenSearch APIs Called
/api/dataconnections/_jobs	POST	Legacy datasourcemanagement.runDirectQuery
/api/dataconnections/_jobs/{queryId}/{dataSourceMDSId?}	GET	Legacy datasourcemanagement.getJobStatus
/api/dataconnections/_jobs/{queryId}	DELETE	Legacy datasourcemanagement.deleteJob

Note: This plugin heavily uses the legacy client pattern with custom OpenSearch plugin endpoints.

Plugin: Chat (ML/AI)

Source: src/plugins/chat/server/routes/ml_routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/chat/memory/sessions/search	POST	transport.request -> ML agent APIs	Search agent memory sessions
/api/chat/proxy	POST	transport.request -> /_plugins/_ml/agents/{id}/_execute	Proxy chat/agent requests via ML/AG-UI

Request/Response Transformations

```

OSD Request: POST /api/chat/proxy
  body: { messages: [...], conversationId: "..." }
  |
  v
generic_ml_router.ts:
  client.transport.request({
    method: 'POST',
    path: '/_plugins/_ml/agents/{agentId}/_execute',
    body: { parameters: { messages, conversationId } }
  })
  |
  v
OpenSearch: POST /_plugins/_ml/agents/{agentId}/_execute
  |
  v
OSD Response: { messages: [...response messages...], conversationId }

```

Plugin: Workspace

Source: src/plugins/workspace/server/routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/workspaces/_list	POST	search (via saved objects client)	List workspaces
/api/workspaces/{id}	GET	get (via saved objects client)	Get workspace by ID
/api/workspaces	POST	index (via saved objects client)	Create workspace
/api/workspaces/{id}	PUT	update (via saved objects client)	Update workspace
/api/workspaces/{id}	DELETE	delete + updateByQuery (via saved objects client)	Delete workspace and dissociate objects
/api/workspaces/_associate	POST	bulk (via saved objects client)	Associate saved objects with workspace
/api/workspaces/_dissociate	POST	bulk (via saved objects client)	Dissociate saved objects from workspace
/api/workspaces/_duplicate	POST	bulk (via saved objects client)	Duplicate saved objects between workspaces

Note: Workspace routes interact with OpenSearch indirectly through the saved objects client.

Plugin: Home (Sample Data)

Source: src/plugins/home/server/services/sample_data/routes/, src/plugins/home/server/routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/sample_data	GET	(none)	List available sample datasets
/api/sample_data/{id}	POST	indices.create + bulk + indices.delete	Install sample dataset (deletes old, creates index, bulk inserts)
/api/sample_data/{id}	DELETE	indices.delete	Uninstall sample dataset
/api/home/hits_status	POST	search	Check if OpenSearch indices have data

/api/opensearch-dashboards/home/tutorials	GET	(none)	Get available tutorials
---	-----	--------	-------------------------

Plugin: Telemetry

Source: src/plugins/telemetry/server/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/telemetry/v2/clusters/_stats	POST	cluster.stats + nodes.usage + info() + indices.getMapping + indices.stats	Comprehensive cluster telemetry collection
/api/telemetry/v2/clusters/_opt_in_stats	POST	cluster.stats (via collector)	Opt-in telemetry stats
/api/telemetry/v2/optIn	POST	(none - saved objects only)	Update telemetry opt-in status
/api/telemetry/v2/userHasSeenNotice	GET	(none - saved objects only)	Check if user has seen telemetry notice

Plugin: Vis Type Timeline

Source: src/plugins/vis_type_timeline/server/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/timeline/run	POST	search (via chain runner)	Execute Timeline expressions against OpenSearch
/api/timeline/validate/opensearch	POST	search (via chain runner)	Validate OpenSearch connection via Timeline
/api/timeline/functions	GET	(none)	Get available Timeline functions

Plugin: Vis Type Timeseries (TSVB)

Source: src/plugins/vis_type_timeseries/server/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/metrics/vis/data	POST	search (via search strategies)	Get TSVB visualization data
/api/metrics/vis/data-raw	POST	search (via search strategies)	Get raw TSVB visualization data
/api/metrics/fields	GET	Field caps (via index patterns)	Get fields for index pattern

Plugin: Data Source

Source: src/plugins/data_source/server/routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/internal/data-source-management/validate	POST	info() or cat.indices	Test data source connection (uses info() for standard, cat.indices for serverless)

/internal/data-source-management/fetchDataSourceMetaData	POST	info()	Fetch data source metadata (version, distribution)
--	------	--------	--

Plugin: Saved Objects Management

Source: src/plugins/saved_objects_management/server/routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/opensearch-dashboards/management/saved_objects/_find	GET	search (via saved objects client)	Find saved objects for management UI
/api/opensearch-dashboards/management/saved_objects/{type}/{id}	GET	get (via saved objects client)	Get saved object
/api/opensearch-dashboards/management/saved_objects/relationships/{type}/{id}	GET	search (via saved objects client)	Get object relationships
/api/opensearch-dashboards/management/saved_objects/_allowed_types	GET	(none)	Get allowed saved object types
/api/opensearch-dashboards/management/saved_objects/scroll/counts	POST	search (via saved objects client)	Get scroll counts
/api/opensearch-dashboards/management/saved_objects/scroll/export	POST	search + scroll (via saved objects client)	Export saved objects via scroll

Plugin: Share (Short URLs)

Source: src/plugins/share/server/routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/short_url	POST	index (via saved objects client)	Create short URL
/api/short_url/{id}	GET	get (via saved objects client)	Resolve short URL

Plugin: Legacy Export

Source: src/plugins/legacy_export/server/routes/

OSD Endpoint	Method	OpenSearch APIs Called	Data Flow
/api/opensearch-dashboards/dashboards/export	GET	search (via saved objects client)	Export dashboards (legacy format)
/api/opensearch-dashboards/dashboards/import	POST	bulk (via saved objects client)	Import dashboards (legacy format)

Core: Dynamic Config Store (Internal)

Source: src/core/server/config/service/config_store_client/opensearch_config_store_client.ts

This is an internal service (no direct HTTP routes) that stores dynamic configuration in OpenSearch.

Internal Operation	OpenSearch APIs Called	Purpose
Initialize config index	indices.existsAlias + indices.create + indices.updateAliases	Creates config index with alias on startup
List configs	search	Retrieves all config documents
Bulk create/update	bulk	Batch config operations
Delete configs	deleteByQuery	Removes configs by name
Find latest index	cat.indices	Finds the most recent config index version

Core: Saved Objects Migrations (Internal)

Source: src/core/server/saved_objects/migrations/core/

Migrations run at startup and are not exposed via HTTP API.

Migration Phase	OpenSearch APIs Called	Purpose
1. Read existing index	indices.get	Get current index mappings and settings
2. Check aliases	indices.getAlias	Determine current index alias state
3. Create new index	indices.create	Create target index with updated mappings
4. Read all docs	search + scroll + clearScroll	Read all documents from source index
5. Count docs	count	Check if migrations are up to date
6. Write migrated docs	bulk	Bulk write transformed documents to new index
7. Switch alias	indices.updateAliases	Atomically switch alias to new index
8. Delete old index	indices.delete	Remove old index
9. Refresh	indices.refresh	Refresh new index
10. Cleanup templates	indices.deleteTemplate	Remove obsolete templates
11. Reindex (if needed)	reindex + tasks.get	For alias conversion, reindex with wait
12. Remove old types	deleteByQuery	Remove obsolete saved object types

Cross-Reference: OpenSearch API to OSD Endpoints

OpenSearch API	OSD Endpoints That Use It
POST /{index}/_search	Saved Objects _find, Data Search, Value Suggestions, Region Map search, Index Pattern preview, Application Config list, Dynamic Config list, Home hits_status, Timeline run, TSVB vis/data, Saved Objects Management, Telemetry
POST /_msearch	Data Search (multi-search), DSL routes
POST /_search/scroll	Saved Objects export, Migrations, Saved Objects Management export
DELETE /_search/scroll	Migrations (clearScroll)
PUT /{index}/_doc/{id}	Saved Objects create/update, Data Importer, Application Config update

PUT /{index}/_create/{id}	Saved Objects create (no-overwrite)
POST /_bulk	Saved Objects bulk ops, Sample Data install, Migrations, Dynamic Config
POST /_mget	Saved Objects get/bulk_get/delete/bulk_create/bulk_update
GET /{index}/_doc/{id}	Saved Objects get, Application Config get
POST /{index}/_update/{id}	Saved Objects update
DELETE /{index}/_doc/{id}	Saved Objects delete, Application Config delete
POST /{index}/_delete_by_query	Migrations cleanup, Dynamic Config delete, Workspace delete
POST /{index}/_update_by_query	Saved Objects deleteByNamespace/deleteByWorkspace
POST /{index}/_count	Migrations (up-to-date check)
POST /_reindex	Migrations (alias conversion)
PUT /{index} (create)	Migrations, Dynamic Config, Data Importer, Sample Data
DELETE /{index}	Migrations, Sample Data uninstall
GET /{index}	Migrations
HEAD /{index}	Data Importer preview/import
POST /_aliases	Migrations, Dynamic Config, Data Importer
GET /{index}/_alias/{name}	Migrations, Dynamic Config
HEAD /_alias/{name}	Dynamic Config
GET /{index}/_mapping	Data Importer preview, Region Map mappings, DSL routes, Telemetry
GET /{index}/_settings	DSL routes
PUT /{index}/_mapping	(archiver internal only)
PUT /{index}/_settings	(archiver internal only)
POST /{index}/_refresh	Migrations
DELETE /_template/{name}	Migrations
GET / (info())	Telemetry cluster info, Data Source validation, Usage Collection
GET /_cluster/state/{metric}	Core health check (internal)
GET /_cluster/stats	Telemetry collection
GET /_cluster/settings	Data Importer
GET /_cat/indices	Data Importer, Region Map, Dynamic Config, Core Usage Data, Data Source validation (serverless)
GET /_cat/plugins	Core cross-compatibility (internal)
GET /_nodes/{node_id}/{metric}	Core version check (internal)
GET /_nodes/usage	Telemetry collection
GET /{index}/_stats	Telemetry data telemetry
GET /_tasks/{task_id}	Migrations (reindex polling)
transport.request (arbitrary)	Console proxy (any), Chat ML APIs, Query Assist ML agents, Data Source connections
Legacy ppl.*	Data Source Management (PPL queries, connections)
Legacy datasourcemangement.*	Data Source Management (direct queries, jobs)

Summary Statistics

Metric	Count
Total OSD API routes	~130+
Routes calling OpenSearch	~95 (~73%)
Unique OpenSearch APIs used	38
Document APIs	15 (search, msearch, scroll, clearScroll, index, create, bulk, get, mget, update, delete, delet
Index Management APIs	13 (indices.create/delete/get/exists/existsAlias/getAlias/updateAliases/putSettings/putMapping
Cluster/Cat/Node APIs	8 (cluster.state, cluster.stats, cluster.getSettings, cat.indices, cat.plugins, nodes.info, nodes
Index Stats API	1 (indices.stats)
Task APIs	1 (tasks.get)
Transport (arbitrary)	1 (transport.request for custom plugin APIs)
Legacy custom endpoints	9 (ppl., <i>datasourcemanagement.</i> , enhancements.*)

Heaviest OpenSearch API Consumers

1. **Saved Objects Repository** -- search, index, create, get, bulk, mget, update, delete, updateByQuery, deleteByQuery (10 APIs)
2. **Saved Objects Migrations** -- 15 different APIs (indices.*, search, scroll, clearScroll, count, bulk, reindex, tasks.get, deleteByQuery)
3. **Dynamic Config Store** -- search, bulk, deleteByQuery, indices.*, cat.indices (8 APIs)
4. **Console Proxy** -- transport.request (can call ANY OpenSearch API)
5. **Telemetry** -- cluster.stats, nodes.usage, info, indices.getMapping, indices.stats (5 APIs)
6. **Data Importer** -- indices.exists, indices.create, index, indices.getMapping, indices.updateAliases, cat.indices, cluster.getSettings (7 APIs)
7. **Data Source Management** -- 7+ legacy endpoints + standard indices/search APIs via DSL routes

Routes with NO OpenSearch interaction

Many OSD API routes serve static configuration, manage UI state, or delegate to other OSD services without directly calling OpenSearch:

- /api/status (reads cached status)
- /api/telemetry/v2/optIn (saved objects only)
- /api/opensearch-dashboards/scripts/languages (static config)
- /api/console/opensearch_config (local config)
- /api/timeline/functions (function registry)

- /api/opensearch-dashboards/management/saved_objects/_allowed_types (type registry)
- /api/opensearch-dashboards/home/tutorials (static content)
- /api/banner/content (config only)
- Authentication/session routes (handled by security plugin)